Group 76 Final Report: Classification of Retinal Diseases Using Optical Coherence Tomography (OCT) Images

Elite Lu, Ishpreet Nagi, Kenneth Ong
{lue13,nagii,salimk4}@mcmaster.ca

1 Introduction

Healthcare providers deal with many vision related illnesses such as Diabetic Macular Edema (DME), Choroidal Neovascularization (CNV), and drusen. The utilization of these retinal scans detect and determine these conditions. An issue that arises is the uniqueness of each ailment, as each condition varies in subtle ways. Correctly identifying retinal diseases can prove to be a challenge in a plethora of cases and with approximately 30 million OCT scans performed each year, this process can be time-consuming and labor-intensive for healthcare professionals. By having a classification model that can help speed up this process, healthcare professionals should be able to focus their attention on helping more individuals requiring their aid.

2 Related Work

With the rising popularity of Machine Learning (ML) and Deep Learning (DL) in recent years, it is not surprising that many researchers around the world have worked on the classification of retinal diseases using ML and/or DL methods. Most related works have approached the problem by incorporating transfer learning to perform multi-class or binary classification [Kermany et al., 2018, Miladinović et al., 2024, Reddy, 2025].

As seen in the notebook created by the Kaggle user Badduri Nagendra Reddy [2025], the fine-tuned model based on ResNet50 was able to achieve an impressive 98.97% final test accuracy on the same dataset as the one used in this project. Similarly, the models defined in the papers typically build upon a pre-trained model, where the authors only unfreeze the weights of the last few layers and add a final classification layer [Kermany et al., 2018, Miladinović et al., 2024].

3 Dataset

3.1 Folder Structure

When downloading the data, the dataset was partitioned as follows:

```
Dataset
├── test
│   ├── CNV
│   ├── DME
│   ├── DRUSEN
│   └── NORMAL
├── train
│   ├── CNV
│   │   └── augmented
│   ├── DME
│   │   └── augmented
│   ├── DRUSEN
│   │   └── augmented
│   └── NORMAL
│       └── augmented
└── val
    ├── CNV
    ├── DME
    ├── DRUSEN
    └── NORMAL
```

The new added folders for the training data, augmented, contains the augmented image data for each category. The reason for this will be elaborated in the **Data Augmentation** section.

3.2 Initial Image Counts

Type	Disease	# of Images
train	CNV	37205
	DME	11348
	DRUSEN	8616
	NORMAL	26315
test	CNV	242
	DME	242
	DRUSEN	242
	NORMAL	242
val	CNV	8
	DME	8
	DRUSEN	8
	NORMAL	8

3.3 Data Augmentation

One of the encountered problems with the dataset is the class imbalance. As seen in the **Initial Image Counts** table, the number of images vary drastically. For example, CNV has nearly 4.5 times the amount of images the DRUSEN class has. Because of this, the model will be biased towards the classes with more images, thus resulting in poor performance. To mitigate this, data augmentation was performed on the training dataset to increase the number of images such that each class has the same number of images. This was done by performing the following random actions:

```
augmentation_transforms = transforms.Compose([
        transforms.RandomHorizontalFlip(),
        transforms.RandomRotation(50),
        transforms.RandomResizedCrop(size=224,
            scale=(0.8, 1.0)),
        transforms.ColorJitter(brightness=0.2,
            contrast=0.2),
    ])
```

This augmentation process includes randomly flipping the image horizontally, randomly rotating the image by up to 50 degrees, randomly cropping and resizing the image, and randomly changing the contrast and brightness of the image. By performing this, not only will the dataset be more robust as a whole, but the model will less likely to overfit or have bias towards a certain class.
As a result, the new image counts after augmentation are as follows:

Type	Disease	# of Images
train	CNV	37205
	DME	37205
	DRUSEN	37205
	NORMAL	37205
test	CNV	242
	DME	242
	DRUSEN	242
	NORMAL	242
val	CNV	8
	DME	8
	DRUSEN	8
	NORMAL	8

This results in the training dataset having a total of 148820 images.

3.4 Preprocessing

Each image was loaded from the various folders as a PIL (Pillow, not the Python Imaging Library) Image and then converted to a tensor object. The pipeline varies slightly for the *train* dataset when compared to both *test* and *val* datasets. Both pipelines resized the images to 224 pixels by 224 pixels and normalized them with means [0.485, 0.456, 0.406] and standard deviation (std) of [0.229, 0.224, 0.225]. Since we are using ResNet50 as the base model, the values for the mean and standard deviation come from ImageNet's preset values, which is the dataset that ResNet is being trained on. The only difference in the preprocessing pipeline is that before normalization, the *train* dataset images might be horizontally flipped whereas the image data for the *test* and *val* pipelines is not. This allowed for the training set to have a more robust dataset. However, this is technically not considered an augmentation since no new data was created (no data distribution changes).

3.5 Annotation

Each image was labeled with an integer to represent its corresponding disease. The images were already named as DISEASE-#####-#.jpeg and located in their corresponding folders, which allowed for ease of labeling. When assigning labels to the dataset, we use the following mapping:

CNV	0
DME	1
DRUSEN	2
NORMAL	3

4 Features

4.1 Data Input

Since this is a computer vision/image classification task, the features were derived from the pixels of the images. Each pixel was denoted by three values: Red(R), Green(G), and Blue(B). As mentioned previously, each image was cropped and normalized with values corresponding the means and standard deviations for images from ImageNet.

4.2 Features

The selection of features was not done directly. This was because it was not required as there are no features to select. Since the pixels are the "feature", this would not be justified. As a result, feature engineering and feature selection was not needed. The same can be said for learning embeddings, as this is not a natural language processing task.
For the output features, there are four main features, which are the four categories of diseases: CNV, DME, DRUSEN, and NORMAL.

5 Implementation

5.1 Learning Model

Instead of training a model from scratch, a pretrained neural network was used and adapted it to the use case. The pretrained neural network selected was ResNet-50, which was trained on the ImageNet dataset. ResNet-50 is a neural network that is 50 layers deep originally developed by Microsoft Research in 2015. This is a type of convolutional neural network (CNN), which is a subset of neural networks that are used for image recognition.

5.2 Implementation Steps

5.2.1 Initializing the Neural Network

To implement the ResNet-50 based neural network, the first step was to instantiate a neural network that uses ResNet-50. This occurs after all the images have been preprocessed. This was called CustomResNet. For the initialization, the input features and classifiers are all defined. The initialization was very similar to typical setup for neural networks.

5.2.2 Freezing and Unfreezing Layers

Upon initialization, the first thing was to freeze all the layers but the last four layers, which includes three feature sequential layers and the classifier layer. This was because a pretrained model such as ResNet-50 already comes with various useful general features such as edges, textures, and colours in early layers. Since the dataset that was used was quite small (four disease classes), training all the layers can result in an increase of time, overfitting, and wiping out pretrained knowledge.

5.2.3 Defining the Criterion and Optimizer

ADD NEW SHIT

5.2.4 Training Data

The dataset has 83484 images in the four folders (CNV, DME, DRUSEN, NORMAL) under training. The distribution was not even, as CNV had significantly more images compared to the other diseases. When comparing CNV to the category with the least images, DRUSEN, CNV has around 2.5 times the images in DRUSEN. This was remedied by augmenting images for each section to match CNV. This totaled 148820 images for model training, with 37205 images per category. By augmenting the images to even out the categories, this prevented skewing and bias.

5.2.5 Validation and Test Data

Since there are significantly fewer images categorized under these, all images were used. There are only 32 images for validation spread evenly in the four diseases. The testing data has an even spread too, with 242 images for each disease, totaling 968 images.

5.2.6 Training and Validation

For training, a total of 100 epochs were performed for the 148820 selected images. The loss and accuracy of both training and validation were printed along with a graph displaying the loss for training and validation.

5.2.7 Testing

After testing, the values were plotted on a 4×4 confusion matrix. The final accuracy of the model was written above it.
ADD NEW SHIT

6 Results and Evaluation

6.1 Training/Validation Phase

After the model was initialized, it was fed through the three stages of training, validation, and testing, with the training and validation phases being combined in a singular loop to save computation

time as well as allow for better performance metric representation. During the training and validation phase, the model's performance was measured via the computation of training/validation loss and training/validation accuracy at each epoch. This allows insight into the performance trend of the model and its overall journey through the training and validation phase. The rate at which the model learns the trends in the dataset can be witnessed in its ability to better classify the images through the loss and accuracy metrics. The loss method was the cross-entropy loss, which is accessed through the nn.CrossEntropyLoss() function that preexists as a part of PyTorch, and the accuracy was calculated via the division of the correct predictions by the model on both the training and validation datasets by the total predictions by the model on the training and validation sets, respectively. In conjunction with this, loss values are stored and calculated for the training and validation steps, and then visualizing the overall trends by plotting them on a line graph to help further understand the model and its performance.

6.2 Test Phase

By this stage, the model has been evaluated once through the validation phase; however, it was evaluated again, this time on the test split of the dataset via the testing phase. For the testing phase, the performance metric of accuracy was used as the evaluation metric of the performance of the model, which was calculated via the same accuracy calculation method as training and validation. In addition to these performance metrics, a confusion matrix was displayed, which visualizes the true positive (TP), true negative (TN), false positive (FP), and false negative (FN) predictions of the model for the testing phase. This aided in easier comprehension and evaluation of the performance of the model on the testing dataset, thus presenting better methods of improvement.

6.3 Quantitative Results

The following results are shown below, which will be acting as the baselines going forward.
ADD NEW SHIT
As seen above, while the model was proceeding as intended, there was a large margin for improvement.

7 Progress

ADD NEW SHIT

8 Error Analysis

ADD NEW SHIT

9 Team Contributions

9.1 Elite

- Worked on the initial image opening and storing of data.
- Wrote sections 1, 3, 4, and 5 of the document.

9.2 Ishpreet

- Wrote the model training code and the ResNet-50 neural network.
- Trained and tested the model.
- Wrote section 6 of the document.

9.3 Kenneth

- Worked on setting up the neural network and research for the start of the project.
- Wrote sections 2 and 7 of the document.

References

Amrit Das, Rohan Giri, Gunjan Chourasia, and A. Anilet Bala. 2019. Classification of retinal diseases using transfer learning approach. In *2019 International Conference on Communication and Electronics Systems (ICCES)*, pages 2080–2084.

Daniel S Kermany, Michael Goldbaum, Wenjia Cai, Carolina C S Valentim, Huiying Liang, Sally L Baxter, Alex McKeown, Ge Yang, Xiaokang Wu, Fangbing Yan, Justin Dong, Made K Prasadha, Jacqueline Pei, Magdalene Y L Ting, Jie Zhu, Christina Li, Sierra Hewett, Jason Dong, Ian Ziyar, and 20 others. 2018. Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5):1122–1131.e9.

Aleksandar Miladinović, Alessandro Biscontin, Miloš Ajčević, Simone Kresevic, Agostino Accardo, Dario Marangoni, Daniele Tognetto, and Leandro Inferrera. 2024. Evaluating deep learning models for classifying OCT images with limited data and noisy labels. *Sci. Rep.*, 14(1):30321.

Badduri Nagendra Reddy. 2025. With casuality. https://www.kaggle.com/code/nagendrareddy777/with-casuality. Accessed: 2025-11-9.

Mehmet Batuhan Özdaş, Fatih Uysal, and Fırat Hardalaç. 2023. Classification of retinal diseases in optical coherence tomography images using artificial intelligence and firefly algorithm. *Diagnostics*, 13(3).